

Київський національний університет імені Тараса Шевченка
Факультет комп'ютерних наук та кібернетики

ЗВІТ

до лабораторного проєкту № 4

з дисципліни *«Операційні системи»*

Тема: Клієнт–серверне сортування масиву

Виконав:
Кіщук Ярослав Ярославович

1. Постановка задачі

Згідно з умовою лабораторного проєкту 4:

- Написати програму-сервер і програму-клієнт для розв'язання задачі сортування масиву. Клієнт формує масив і відправляє на сервер; сервер сортує і відправляє відсортований масив на клієнт.
- На клієнті фіксувати час початку відправлення масиву і час завершення отримання результатів.
- Виконати експеримент 100 разів і визначити середній показник швидкості роботи мережного порту.
- Кількість елементів — 10 000; метод сортування — на вибір (у цій роботі — **QuickSort**, $O(n \log n)$ у середньому).

2. Протокол і архітектура

Транспорт — TCP (127.0.0.1:9504, loopback). Бінарний формат повідомлення наведено в табл. 1.

Таблиця 1 – Формат бінарного повідомлення

Поле	Розмір
N	4 байти, uint32 little-endian
data	$N \times 4$ байти (int32, little-endian)

Обсяг одного round-trip (запит + відповідь): $2 \times (4 + 10000 \times 4) = 80\,008$ байт.

Середня швидкість порту:

$$V = \frac{2 \cdot (4 + N \cdot 4) \cdot 8}{T_{\text{avg}}} \quad (\text{Мбіт/с}),$$

де T_{avg} — середній час від початку відправлення до завершення читання відповіді на клієнті.

3. Код сервера

Listing 1: cmd/server/main.go

```

1 package main
2
3 import (
4     "flag"
5     "log"
6     "net"
7     "os"
8     "os/signal"
9     "sync/atomic"
10    "syscall"
11    "time"
12
13    "lab4/internal/protocol"
```

```

14     sortpkg "lab4/internal/sort"
15 )
16
17 func main() {
18     addr := flag.String("addr", ":9504", "TCP listen address")
19     flag.Parse()
20
21     ln, err := net.Listen("tcp", *addr)
22     if err != nil {
23         log.Fatalf("listen: %v", err)
24     }
25     defer ln.Close()
26
27     var shuttingDown atomic.Bool
28     stop := make(chan os.Signal, 1)
29     signal.Notify(stop, os.Interrupt, syscall.SIGTERM)
30     go func() {
31         <-stop
32         log.Println("shutting down")
33         shuttingDown.Store(true)
34         ln.Close()
35     }()
36
37     log.Printf("server listening on %s", *addr)
38     for {
39         conn, err := ln.Accept()
40         if err != nil {
41             if shuttingDown.Load() {
42                 return
43             }
44             log.Printf("accept: %v", err)
45             continue
46         }
47         go handleConn(conn)
48     }
49 }
50
51 func handleConn(conn net.Conn) {
52     defer conn.Close()
53     remote := conn.RemoteAddr().String()
54
55     data, err := protocol.ReadArray(conn)
56     if err != nil {
57         log.Printf("[%s] read: %v", remote, err)
58         return
59     }
60     log.Printf("[%s] received %d elements", remote, len(data))
61
62     t0 := time.Now()
63     sortpkg.SortInPlace(data)
64     sortDur := time.Since(t0)
65
66     if err := protocol.WriteArray(conn, data); err != nil {
67         log.Printf("[%s] write: %v", remote, err)
68         return
69     }
70     log.Printf("[%s] sorted and replied (sort %v)", remote, sortDur)
71 }

```

Listing 2: internal/protocol/array.go

```

1 package protocol
2
3 import (
4     "encoding/binary"
5     "fmt"
6     "io"
7 )
8
9 const HeaderSize = 4
10
11 func WriteArray(w io.Writer, data []int32) error {
12     if len(data) > int(^uint32(0)) {
13         return fmt.Errorf("array too large: %d elements", len(data))
14     }
15     header := make([]byte, HeaderSize)
16     binary.LittleEndian.PutUint32(header, uint32(len(data)))
17     if _, err := w.Write(header); err != nil {
18         return err
19     }
20     if len(data) == 0 {
21         return nil
22     }
23     payload := make([]byte, len(data)*4)
24     for i, v := range data {
25         binary.LittleEndian.PutUint32(payload[i*4:], uint32(v))
26     }
27     _, err := w.Write(payload)
28     return err
29 }
30
31 func ReadArray(r io.Reader) ([]int32, error) {
32     header := make([]byte, HeaderSize)
33     if _, err := io.ReadFull(r, header); err != nil {
34         return nil, err
35     }
36     n := binary.LittleEndian.Uint32(header)
37     if n == 0 {
38         return nil, nil
39     }
40     payload := make([]byte, int(n)*4)
41     if _, err := io.ReadFull(r, payload); err != nil {
42         return nil, err
43     }
44     out := make([]int32, n)
45     for i := range out {
46         out[i] = int32(binary.LittleEndian.Uint32(payload[i*4:]))
47     }
48     return out, nil
49 }
50
51 func BytesPerMessage(n int) int64 {
52     return int64(HeaderSize + n*4)
53 }

```

Listing 3: internal/sort/quicksort.go

```

1 package sortpkg
2
3 func SortInPlace(data []int32) {
4     if len(data) < 2 {
5         return

```

```

6     }
7     quickSort(data, 0, len(data)-1)
8 }
9
10 func quickSort(data []int32, lo, hi int) {
11     for lo < hi {
12         p := partition(data, lo, hi)
13         if p-lo < hi-p {
14             quickSort(data, lo, p-1)
15             lo = p + 1
16         } else {
17             quickSort(data, p+1, hi)
18             hi = p - 1
19         }
20     }
21 }
22
23 func partition(data []int32, lo, hi int) int {
24     pivot := data[hi]
25     i := lo
26     for j := lo; j < hi; j++ {
27         if data[j] <= pivot {
28             data[i], data[j] = data[j], data[i]
29             i++
30         }
31     }
32     data[i], data[hi] = data[hi], data[i]
33     return i
34 }

```

4. Код клієнта

Listing 4: cmd/client/main.go

```

1 package main
2
3 import (
4     "flag"
5     "fmt"
6     "log"
7     "math"
8     "math/rand"
9     "net"
10    "time"
11
12    "lab4/internal/protocol"
13 )
14
15 func main() {
16     addr := flag.String("addr", "127.0.0.1:9504", "server address")
17     n := flag.Int("n", 10000, "array length")
18     runs := flag.Int("runs", 100, "number of experiments")
19     seed := flag.Int64("seed", 42, "RNG seed base")
20     flag.Parse()
21
22     if *n <= 0 {
23         log.Fatal("n must be positive")
24     }
25     if *runs <= 0 {
26         log.Fatal("runs must be positive")

```

```

27     }
28
29     rtts := make([]time.Duration, *runs)
30     bytesPerRun := 2 * protocol.BytesPerMessage(*n)
31
32     fmt.Printf("Lab 4 client: addr=%s n=%d runs=%d seed=%d\n", *addr, *n, *runs, *seed)
33     fmt.Printf("Bytes per round-trip (request+response): %d\n\n", bytesPerRun)
34
35     for i := 0; i < *runs; i++ {
36         data := randomArray(*n, *seed+int64(i))
37         dur, err := roundTrip(*addr, data)
38         if err != nil {
39             log.Fatalf("run %d: %v", i+1, err)
40         }
41         if !isSorted(data) {
42             log.Fatalf("run %d: server returned unsorted array", i+1)
43         }
44         rtts[i] = dur
45     }
46
47     avg, min, max, std := stats(rtts)
48     avgSec := avg.Seconds()
49     mbps := float64(bytesPerRun) * 8 / avgSec / 1e6
50
51     fmt.Printf("=== Results (%d experiments) ===\n", *runs)
52     fmt.Printf("RTT avg: %v\n", avg)
53     fmt.Printf("RTT min: %v\n", min)
54     fmt.Printf("RTT max: %v\n", max)
55     fmt.Printf("RTT stdev: %v\n", std)
56     fmt.Printf("Average port speed: %.2f Mbit/s\n", mbps)
57     fmt.Printf("Average port speed: %.2f MB/s\n", float64(bytesPerRun)/avgSec/1e6)
58 }
59
60 func roundTrip(addr string, data []int32) (time.Duration, error) {
61     conn, err := net.Dial("tcp", addr)
62     if err != nil {
63         return 0, err
64     }
65     defer conn.Close()
66
67     t0 := time.Now()
68     if err := protocol.WriteArray(conn, data); err != nil {
69         return 0, err
70     }
71     sorted, err := protocol.ReadArray(conn)
72     if err != nil {
73         return 0, err
74     }
75     t1 := time.Now()
76
77     copy(data, sorted)
78     return t1.Sub(t0), nil
79 }
80
81 func randomArray(n int, seed int64) []int32 {
82     rng := rand.New(rand.NewSource(seed))
83     out := make([]int32, n)
84     for i := range out {
85         out[i] = rng.Int31()
86     }

```

```

87     return out
88 }
89
90 func isSorted(data []int32) bool {
91     for i := 1; i < len(data); i++ {
92         if data[i] < data[i-1] {
93             return false
94         }
95     }
96     return true
97 }
98
99 func stats(rtts []time.Duration) (avg, min, max, std time.Duration) {
100     if len(rtts) == 0 {
101         return
102     }
103     min, max = rtts[0], rtts[0]
104     var sum int64
105     for _, d := range rtts {
106         sum += d.Nanoseconds()
107         if d < min {
108             min = d
109         }
110         if d > max {
111             max = d
112         }
113     }
114     avg = time.Duration(sum / int64(len(rtts)))
115     var varSum float64
116     avgNs := float64(avg.Nanoseconds())
117     for _, d := range rtts {
118         diff := float64(d.Nanoseconds()) - avgNs
119         varSum += diff * diff
120     }
121     std = time.Duration(math.Sqrt(varSum / float64(len(rtts))))
122     return
123 }

```

5. Результати випробувань

Платформа: macOS, Apple Silicon, loopback TCP. Експеримент: ./examples/run_experiment.sh (100 прогонів, seed=42).

Таблиця 2 – Метрики round-trip (100 експериментів)

Показник	Значення
Середній RTT	530.293 μ s
Мінімальний RTT	435.125 μ s
Максимальний RTT	868.375 μ s
Стандартне відхилення	78.105 μ s
Середня швидкість порту	1207.00 Мбіт/с
Пропускна здатність	150.88 МБ/с

```
server
$ ./bin/server -addr 127.0.0.1:9504
2026/05/18 server listening on 127.0.0.1:9504
2026/05/18 [127.0.0.1:51440] received 10000 elements
2026/05/18 [127.0.0.1:51440] sorted and replied (sort 512Bs)
2026/05/18 [127.0.0.1:51441] received 10000 elements
2026/05/18 [127.0.0.1:51441] sorted and replied (sort 445Bs)
... (100 connections total)
```

Рис. 1: Фрагмент журналу сервера

```
client
$ ./bin/client -runs 100
Lab 4 client addr=127.0.0.1:9504 n=10000 runs=100 seed=42
Bytes per round-trip (request+response): 80008

=== Results (100 experiments) ===
RTT avg: 530.293Bs
RTT min: 435.125Bs
RTT max: 868.375Bs
RTT stdev: 78.105Bs
Average portspeed: 1207.00 Mbit/s
Average portspeed: 150.88 MB/s
```

Рис. 2: Підсумок клієнта після 100 експериментів

6. Висновки

- Реалізовано клієнт–серверну систему на Go з TCP і бінарним протоколом; усі 100 прого-нів повернули коректно відсортовані масиви.
- На loopyback домінує низька затримка; виміряна швидкість порту (≈ 1.2 Гбіт/с) відображає ефективну передачу ≈ 80 КБ за ≈ 0.5 мс, включно з сортуванням на сервері (≈ 0.4 – 0.8 мс).
- QuickSort на 10 000 елементів не є вузьким місцем порівняно з накладними витратами встановлення TCP-з'єднання на кожній ітерації.